

IT5437 — Image Processing and Machine Vision

Vidanapathirana K.D. | Index: 258839R

Assignment 1: Intensity Transformations and Neighborhood Filtering

GITHUB REPOSITORY: https://github.com/Kalana-Demintha-Vidanapathirana/258839R_IT5437_CV_Assignment_1.git

Question 1 — Intensity Transformations on the Runway Image

Key code

```
def gamma_transform(image_norm, gamma):
    return np.power(image_norm, gamma)

def contrast_stretch(image_norm, r1=0.2, r2=0.8):
    out = np.zeros_like(image_norm, dtype=np.float32)
    out[image_norm < r1] = 0.0
    out[(image_norm >= r1) & (image_norm <= r2)] = (image_norm[(image_norm >= r1) & (image_norm <= r2)] - r1) / (r2 - r1)
    out[image_norm > r2] = 1.0
    return out
```

Results Interpretation

Original	$\gamma = 0.5$ (brighten)	$\gamma = 2.0$ (darken)	Contrast Stretch ($r_1=0.2, r_2=0.8$)
			

Discussion

- $\gamma = 0.5$ raises every normalised pixel toward 1 — dark areas (tarmac, shadows) brighten; the sky saturates toward white.
- $\gamma = 2.0$ pushes midtones into shadows, revealing sky texture while crushing dark runway detail.
- Contrast stretching maps the mid-band $[0.2, 0.8]$ to the full $[0, 1]$, maximising perceived contrast for this histogram-concentrated scene.

Question 2 — Gamma Correction in $L^*a^*b^*$ Colour Space

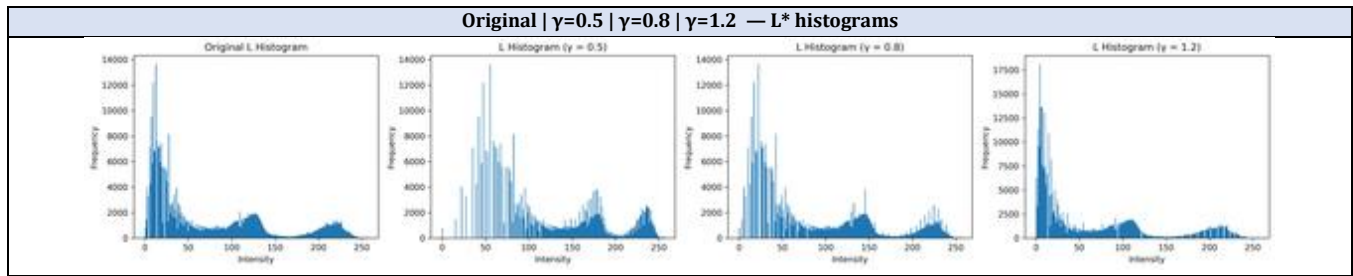
Key code

```
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(img_lab)
L_norm = L.astype(np.float32) / 255.0
L_corr = np.clip(np.power(L_norm, gamma) * 255, 0, 255).astype(np.uint8)
result = cv2.cvtColor(cv2.merge([L_corr, a, b]), cv2.COLOR_LAB2BGR)
# Applied with  $\gamma = 0.8$  (brightening)
```

Results Interpretation

Original	L^* gamma-corrected ($\gamma = 0.8$)
	

Histograms (L channel)



Discussion

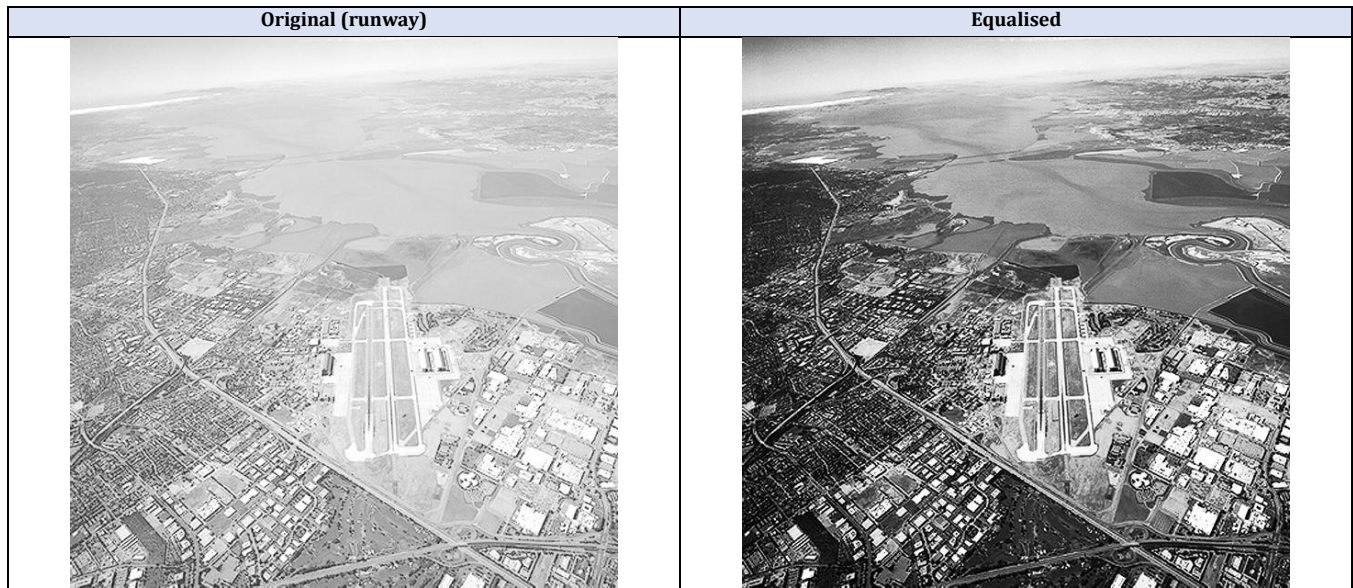
- Correction is applied only to the L* channel, so colours (a^* , b^*) are preserved exactly.
- $\gamma = 0.8 < 1$ shifts the histogram rightward — pixels move to higher intensities without colour distortion.
- $\gamma = 0.5$ brightens aggressively; $\gamma = 1.2$ darkens slightly. $\gamma = 0.8$ gives the best balance for this scene.

Question 3 — Manual Histogram Equalization

Key code

```
def histogram_equalize(img):
    hist = np.bincount(img.flatten(), minlength=256)
    cdf = hist.cumsum()
    cdf_min = cdf[np.nonzero(cdf)][0]
    lut = np.round((cdf - cdf_min) / (img.size - cdf_min) * 255)
    return np.clip(lut, 0, 255).astype(np.uint8)[img]
```

Result Interpretation



Discussion

- Equalization maps intensities via the normalized CDF, redistributing them toward a uniform distribution.
- The mapping is monotonic — relative pixel brightness is preserved; previously compressed intensity bands are spread out.
- Unlike gamma correction, no parameter tuning is required; the transform adapts fully to the image histogram.

Question 4 — Otsu Thresholding + Foreground Histogram Equalization

Key code

```
otsu_thresh, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
if gray[mask==255].mean() > gray[mask==0].mean(): # keep darker region as foreground
    mask = cv2.bitwise_not(mask)
fg = gray[mask == 255]
hist = np.bincount(fg, minlength=256); cdf = hist.cumsum()
lut = np.round((cdf - cdf[cdf>0][0]) / (fg.size - cdf[cdf>0][0]) * 255)
result[mask == 255] = np.clip(lut, 0, 255).astype(np.uint8)[gray[mask == 255]]
```

Otsu threshold value = 101

Results Interpretation

Grayscale	Otsu Mask	Foreground Equalized
		

Discussion

- Otsu's method (threshold = 101) automatically separates the bright window/door from the darker woman and interior.
- Equalization applied only to the foreground reveals clothing texture and room detail without over-saturating the already bright window.
- Background pixels remain unchanged, preserving the natural contrast between interior and exterior.

Question 5 — Manual Gaussian Filtering

Key code

```
def gaussian_kernel(size, sigma):
    ax = np.arange(-(size // 2), size // 2 + 1)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
    return kernel / kernel.sum() # normalized — sums to 1

smoothed_manual = cv2.filter2D(img, -1, gaussian_kernel(5, 2))
smoothed_cv = cv2.GaussianBlur(img, (5, 5), sigmaX=2)
```

Results Interpretation

Original	Manual (5×5, $\sigma=2$)	OpenCV GaussianBlur
		

Discussion

- The normalized kernel (sum = 1) preserves mean brightness after filtering.
- Manual filter2D and OpenCV GaussianBlur produce near-identical results, confirming implementation correctness.
- Minor pixel-level differences arise from boundary handling; for $\sigma=2$ a 5×5 kernel is slightly small to capture the full Gaussian spread.

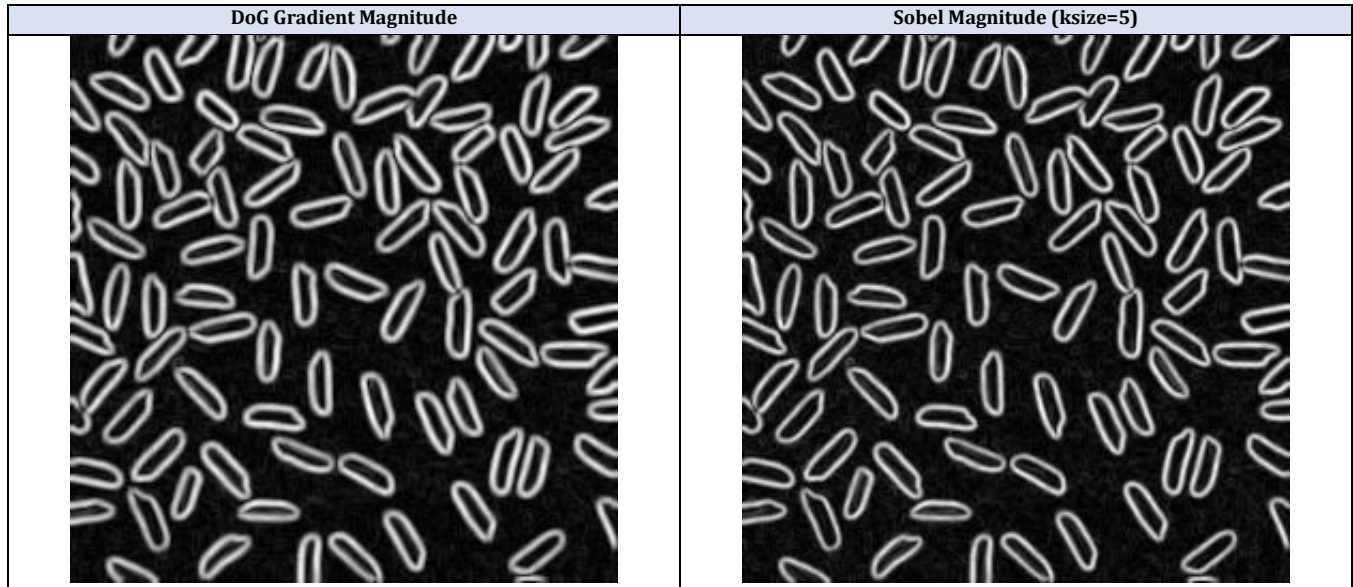
Question 6 — Derivative of Gaussian (DoG) Edge Detection

Key code

```
def dog_kernels(size, sigma):
    ax = np.arange(-(size//2), size//2+1); xx, yy = np.meshgrid(ax, ax)
    G = (1/(2*np.pi*sigma**2)) * np.exp(-(xx**2 + yy**2) / (2*sigma**2))
    Gx = -(xx / sigma**2) * G;    Gy = -(yy / sigma**2) * G
    return Gx / np.sum(np.abs(Gx)), Gy / np.sum(np.abs(Gy))

grad_x = cv2.filter2D(img.astype(np.float32), -1, gx)
grad_y = cv2.filter2D(img.astype(np.float32), -1, gy)
grad_mag = np.sqrt(grad_x**2 + grad_y**2)
# Compared with: cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=5)
```

Results Interpretation



Discussion

- $\partial G / \partial x = -(x/\sigma^2) \cdot G(x,y)$ is odd in x and even in y, giving zero-sum rows — a mathematically valid derivative operator.
- DoG at $\sigma=2$ yields cleaner, thicker, more noise-robust edges than Sobel because Gaussian smoothing pre-suppresses noise.
- σ controls detection scale: small σ finds fine texture edges, large σ finds coarser structural boundaries (principle behind Canny and SIFT).

Question 7 — Image Zooming: Nearest-Neighbour vs. Bilinear

Key code

```
def resize_nearest(img, sx, sy):
    for y_new in range(new_h):
        for x_new in range(new_w):
            out[y_new, x_new] = img[min(round(y_new/sy), h-1), min(round(x_new/sx), w-1)]

def resize_bilinear(img, sx, sy):
    for y_new, x_new in ...:
        y, x = y_new/sy, x_new/sx; y0, x0 = floor(y), floor(x); dy, dx = y-y0, x-x0
        top = (1-dx)*img[y0,x0] + dx*img[y0,x1]
        out[y_new,x_new] = (1-dy)*top + dy*((1-dx)*img[y1,x0] + dx*img[y1,x1])

def normalized_ssd(img1, img2):
    return np.sum((img1.astype(float) - img2.astype(float))**2) / img1.size
```

Quantitative comparison — Normalised SSD vs. original

Image pair	Scale	SSD — Nearest Neighbour	SSD — Bilinear
im01	4×	255.30	200.25
im02	4×	64.63	48.96
im03	4×	147.77	113.57
taylor (small)	5×	320.09	285.21
taylor (very small)	20×	737.81	654.63

Results Interpretation

NN upsample (5×)		Bilinear upsample (5×)	
			

Discussion

- Bilinear SSD is consistently lower across all image pairs, confirming higher reconstruction fidelity.
- Nearest-neighbour preserves crisp edges but produces blocky pixel steps, especially on diagonal and curved features.
- Bilinear interpolates a convex combination of four neighbours, suppressing aliasing at the cost of slight blurring.

Question 8 — Salt-and-Pepper Noise Removal

Key code

```
gaussian_smoothed = cv2.GaussianBlur(img, (5, 5), sigmaX=1.0)
median_filtered = cv2.medianBlur(img, 5)
```

Results Interpretation

Noisy input	Gaussian (5×5, $\sigma=1$)	Median (5×5)
		

Discussion

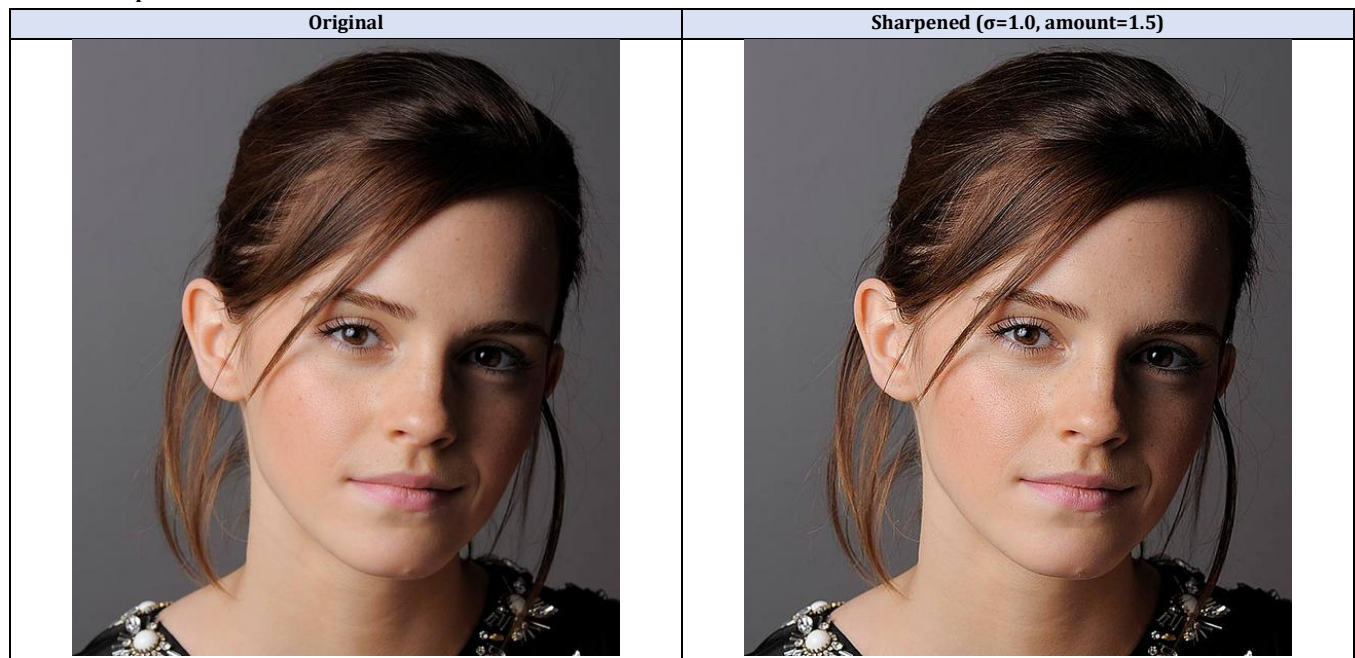
- Salt-and-pepper noise produces isolated extreme pixels; Gaussian smoothing spreads them into blurry grey patches rather than removing them.
- Median filtering replaces each pixel with the neighbourhood median, effectively eliminating impulse noise while preserving edges.
- For impulsive noise, median filtering is clearly superior. Gaussian smoothing is better suited to additive Gaussian noise.

Question 9 — Image Sharpening (Unsharp Masking)

Key code

```
def unsharp_mask(img, kernel_size=(5,5), sigma=1.0, amount=1.5):
    blurred = cv2.GaussianBlur(img, kernel_size, sigmaX=sigma)
    sharpened = cv2.addWeighted(img, 1 + amount, blurred, -amount, 0)
    return blurred, sharpened
# output = (1 + a) · f - a · (f * G) = f + a · (f - blurred)
```

Results Interpretation



Discussion

- Unsharp masking adds a scaled high-pass signal ($f - \text{blurred}$) back to the original, boosting fine detail and edges.
- σ controls the scale of detail enhanced; amount=1.5 provides clear sharpening without excessive halo artefacts.
- Noise is also high-frequency and will be amplified — sharpening should be applied after denoising.

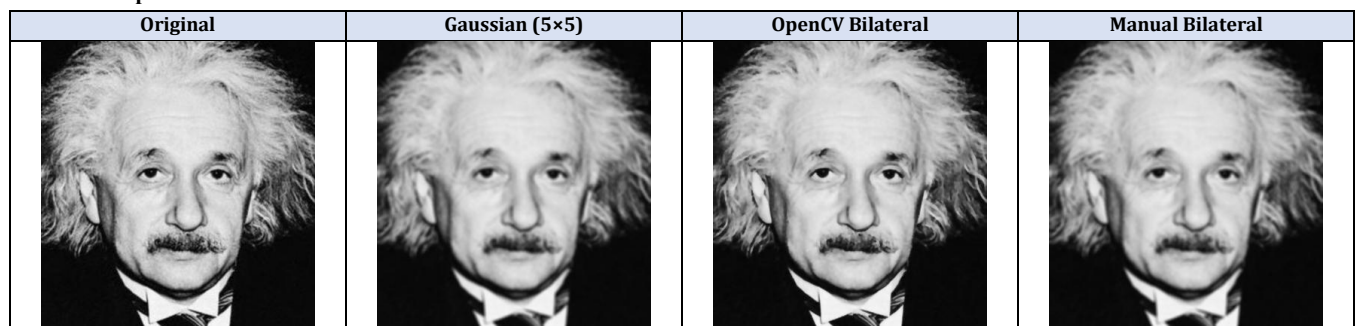
Question 10 — Bilateral Filtering (Edge-Preserving Smoothing)

Key code

```
def manual_bilateral_filter(img, kernel_size=5, sigma_s=3.0, sigma_r=25.0):
    spatial = np.exp(-(xx**2 + yy**2) / (2 * sigma_s**2)) # geometry weight
    for y, x in ...:
        region = padded[y:y+ks, x:x+ks]; centre = padded[y+pad, x+pad]
        range_w = np.exp(-(region - centre)**2 / (2 * sigma_r**2)) # photometric weight
        output[y, x] = np.sum(spatial * range_w * region) / np.sum(spatial * range_w)

opencv_bilateral = cv2.bilateralFilter(img, d=5, sigmaColor=25.0, sigmaSpace=3.0)
```

Results Interpretation



Discussion

- Bilateral filtering weights neighbours by both spatial proximity (σ_s) and intensity similarity (σ_r), preserving edges.
- Gaussian blur smooths uniformly — edges blur. Bilateral maintains sharp edges while smoothing flat/low-contrast regions.
- Manual and OpenCV implementations produce nearly identical results, verifying correctness of the manual filter.

Question 11 — Relationship Between Spatial Filtering and Frequency Response

(a) **Convolution Theorem.** Spatial convolution $g(x,y) = f(x,y) * h(x,y)$ corresponds to pointwise multiplication $G(u,v) = F(u,v) \cdot H(u,v)$ in the frequency domain. Applying a filter spatially is therefore equivalent to multiplying the image spectrum by the filter's transfer function $H(u,v)$.

(b) **Qualitative frequency behaviour.** Averaging (box) filter — low-pass with sinc-shaped frequency response: broad main lobe, oscillating side lobes; blurs and removes high-frequency detail. Gaussian filter — low-pass with Gaussian frequency response: smooth, radially symmetric, no side lobes; gentle noise suppression without ringing. Laplacian filter — high-pass proportional to $-(u^2+v^2)$; amplifies edges and fine detail at the cost of also amplifying noise.

(c) **No ringing in Gaussian.** An ideal low-pass filter has a sharp rectangular cutoff in the frequency domain, whose spatial impulse response is an oscillatory sinc — causing Gibbs/ringing artefacts near edges. A Gaussian is its own Fourier transform: smooth and monotonically decaying in both domains, so no side lobes exist and no ringing arises.

(d) **Best filter for high-frequency noise.** The Gaussian filter is most suitable: it attenuates high-frequency noise gradually as a smooth low-pass with no ringing. The Laplacian amplifies high-frequency noise. The box filter also suppresses high frequencies but its sinc transfer function introduces side-lobe artefacts.

Question 12 — Illumination Correction via Homomorphic Filtering

(a) **Multiplicative model.** $f(x,y) = i(x,y) \cdot r(x,y)$ separates illumination i (light falling on the scene) from reflectance r (surface albedo). Illumination is low-frequency (slow spatial variation — shadows, vignetting); reflectance is high-frequency (texture, edges). Uneven lighting multiplies intensity values — simple brightness adjustment cannot fully correct it.

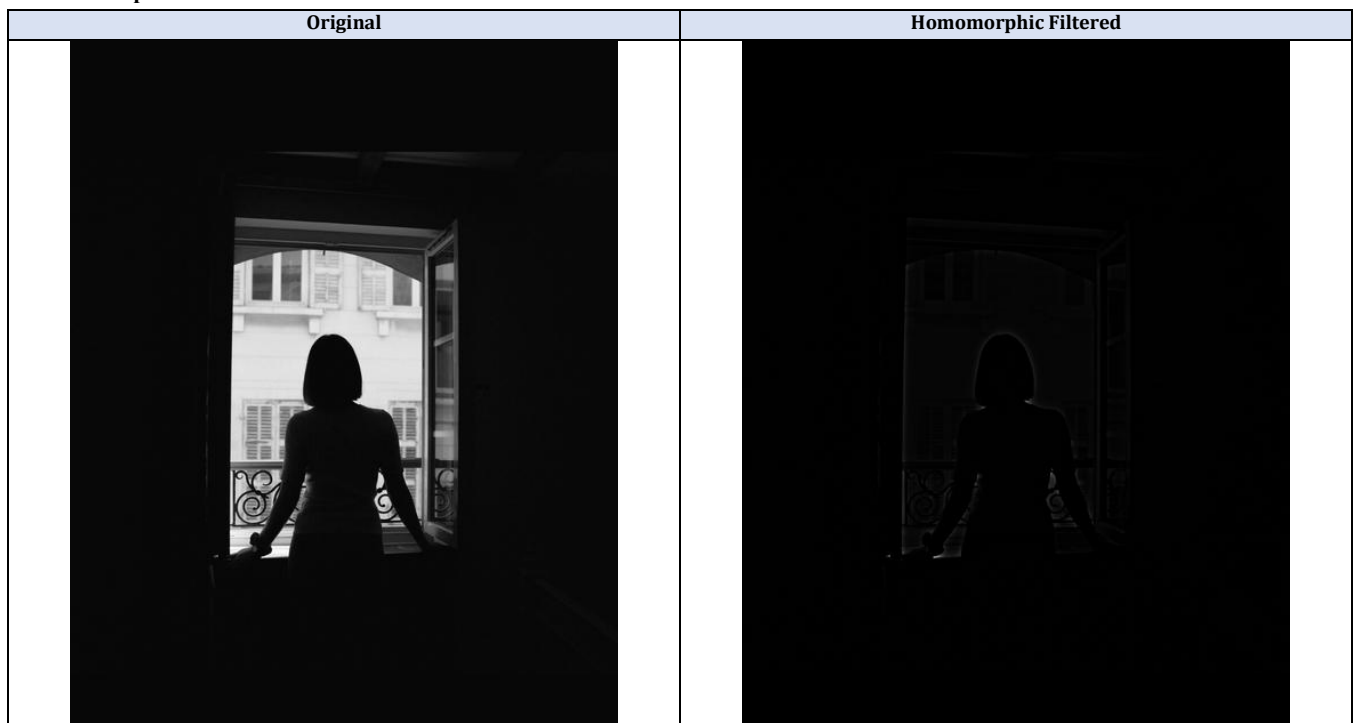
(b) **Log transform.** Taking the natural log: $\ln f = \ln i + \ln r$ converts the multiplicative model into an additive one. Linear frequency-domain filtering can then separately attenuate $\ln i$ (low-frequency illumination) and preserve or boost $\ln r$ (high-frequency reflectance).

(c) **Algorithm.** $\log \rightarrow \text{FFT} \rightarrow \text{multiply by } H(u,v) = (\gamma_H - \gamma_L)(1 - \exp(-c \cdot D^2/D_0^2)) + \gamma_L \rightarrow \text{IFFT} \rightarrow \exp$. With $\gamma_L < 1 < \gamma_H$ the filter attenuates low-frequency illumination and boosts high-frequency reflectance/texture.

Key code

```
def homomorphic_filter(img, gamma_l=0.6, gamma_h=1.8, c=1.0, d0=30):
    log_img = np.log1p(img.astype(np.float32))
    fft_shift = np.fft.fftshift(np.fft.fft2(log_img))
    D2 = (y - crow)**2 + (x - ccol)**2 # squared distance from centre
    H = (gamma_h - gamma_l) * (1 - np.exp(-c * D2 / d0**2)) + gamma_l
    out = np.real(np.fft.ifft2(np.fft.ifftshift(H * fft_shift)))
    return cv2.normalize(np.expml(out), None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
# Parameters:  $\gamma_L = 0.6$ ,  $\gamma_H = 1.8$ ,  $c = 1$ ,  $D_0 = 30$ 
```

Results Interpretation



(d) Comparison with histogram equalization.

Aspect	Histogram Equalization	Homomorphic Filtering
Model assumption	None — global CDF redistribution	Multiplicative $f = i \cdot r$
Domain	Spatial, pixel-wise	Log + frequency domain
Handles non-uniform light?	Poorly — global effect	Directly flattens slow illumination
Fine detail enhancement	Side effect only	Explicit high-frequency boost
Failure modes	Over-stretches noise	Halos if params too aggressive

(e) Observations on woman_window.jpg. Illumination correction: the brightness gradient between the dark interior and bright window is reduced — shadows lift, highlights come down. Contrast enhancement: reflectance detail (woman's clothing, door grain, room boundaries) becomes more visible. Artefacts: slight noise amplification in previously flat dark regions; halo-like fringes near strong edges if parameters are too aggressive. Homomorphic filtering is preferable to histogram equalization here because the dominant problem is spatially non-uniform illumination, not globally compressed contrast.